Iris recognition project documentation

Martyna Sadowska, Hanna Szczerbińska

1 Introduction

Our iris recognition application enables the generation of iris codes from standard eye photographs. A key feature is visually demonstrating each processing stage needed to achieve the final result. Moreover, the application allows for the direct comparison of two separate eye photos by analyzing their corresponding codes. Consequently, it proves to be a reliable solution for human identification.

The project is a desktop application implemented in Java. The Graphical User Interface is built using Swing, which provides a reliable, native toolkit. To ensure a modern appearance, the FlatLaf (FlatDarkLaf) library was utilized to apply a dark theme. The complete source code is available on GitHub. All photos used for testing are from the MMU Iris Dataset.

2 Iris recognition

This panel allows for the visualization of each step required to generate the final iris code. Initially, the user can choose to work with a default testing image or import a chosen photo. Once the image is loaded, the process can be executed sequentially.

2.1 Convert to grayscale

This step converts the image into shades of gray using the luminance algorithm. The calculated gray value is a weighted average of red, green and blue channels, where the coefficients are taken from BT.601 recommendation: 0.299 for red, 0.587 for green and 0.114 for blue. Figure 1 shows the testing eye image after grayscaling.

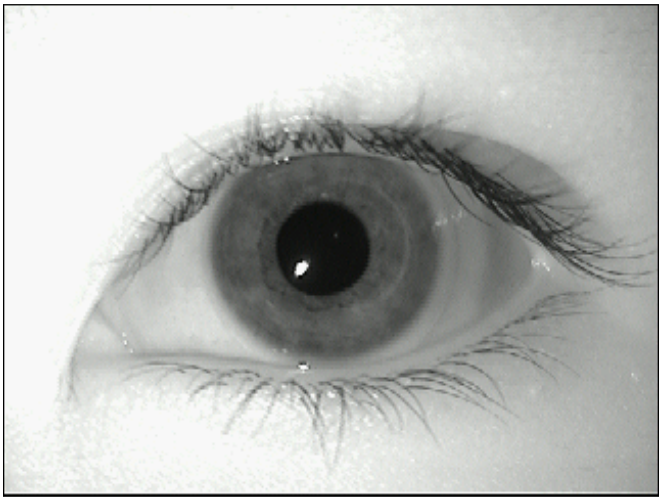

Figure 1: Grayscaled photo used for testing.

2.2 Apply binarization for the pupil

This step applies binarization to isolate the pupil from the rest of the image. The threshold is calculated using the formula

$$\frac{1}{h \cdot w \cdot x} \sum_{i=0}^{h-1} \sum_{j=0}^{w-1} A(i,j),$$

where h is the image height, w is the image width, $A(i,j)$ is the brightness of a specific pixel and x is a threshold divisor. Through empirical testing, the optimal value for the divisor was determined to be 15. The result of this binarization is shown in Figure 2.

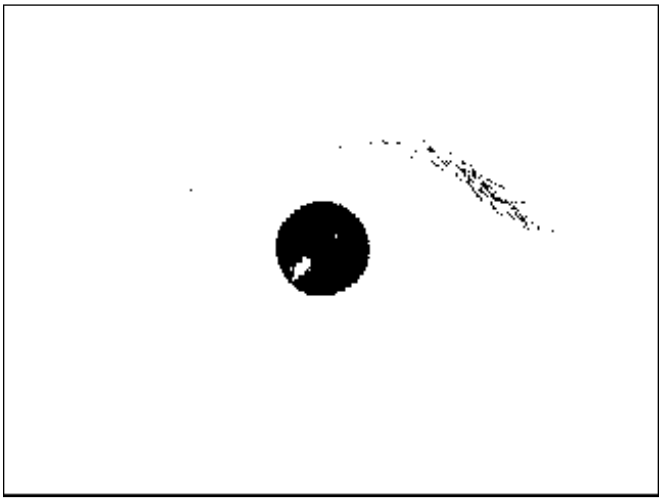

Figure 2: Pupil isolated using binarization.

2.3 Apply morphology operations

After binarization, artifacts such as eyelashes may still be visible, and the pupil region might contain holes caused by light reflections. To correct this, morphology operations that gave the best results are applied. First, the image goes through an opening operation that uses a 5-pixel radius disk as a structuring element to remove noise. Then, a closing operation is applied using a 7-pixel radius disk to fill any remaining gaps. This produces a clean and smooth pupil mask, as seen in Figure 3.

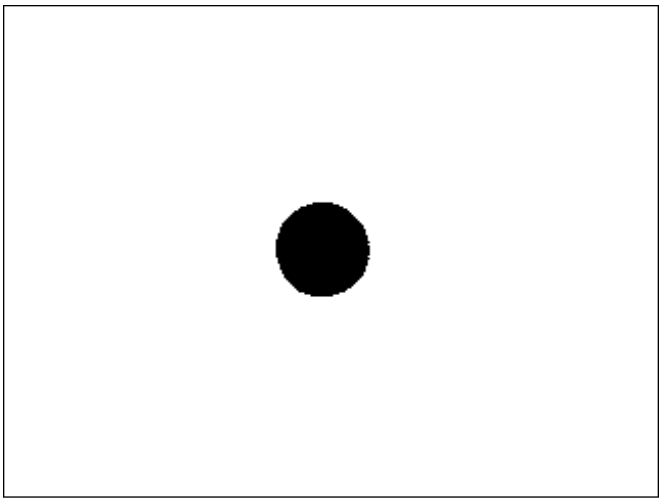

Figure 3: Pupil isolated after applying morphology operations.

2.4 Find the pupil boundaries

The first parameter calculated to determine the boundaries is the pupil center, which will also serve as the center for the iris. This is achieved by analyzing the vertical and horizontal projections of the image, representing the distribution of black pixels along the rows and columns. The X and Y coordinates of the eye center correspond to the indices with the greatest projection values. If the maximum value spans across multiple pixels, the midpoint between the first and last pixel in that range is used. After that the algorithm traces the full width and height of the pupil from this peak to its edges. The final center is calculated as the exact midpoint between the pupil's edges.

To prevent peripheral noise (such as hair) from skewing the results, the outer margins (the first and last 25% of the image's height and width) are excluded from the projection calculations. This change was introduced with the assumption that the eye is around the center of the image. This proved safer than assuming that earlier binarization and morphology operations would get rid of dark hair at the margins of the image.

The second parameter is the pupil radius, which is calculated as the average distance from the newly found center's X-coordinate to the left and right pupil boundaries, determined using the vertical projection.

Once computed, the eye center and the pupil boundary are visualized on the binarized image, as illustrated in Figure 4.

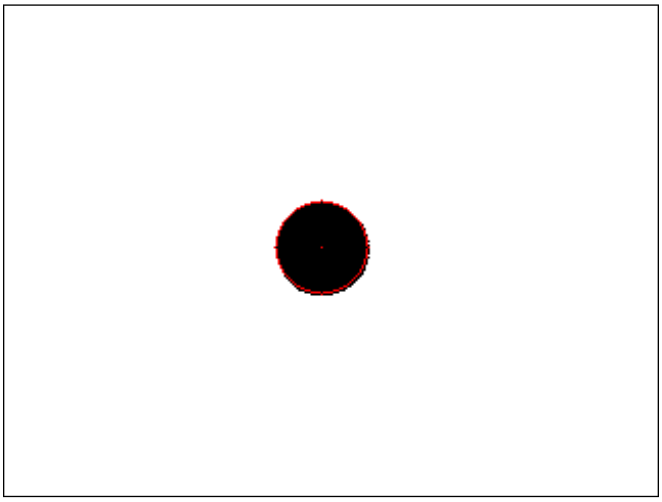

Figure 4: Calculated pupil center and boundaries.

2.5 Revert to grayscaled image

This action reverts the currently displayed image back to the previously generated grayscale version, providing the necessary input for the subsequent boundary detection steps.

2.6 Find the iris boundaries

This step calculates the iris boundaries using a 1D simplification of Daugman's integro-differential operator. To avoid interference from eyelashes and eyelids, the algorithm evaluates concentric circular contours starting just outside the pupil boundary, exclusively analyzing the left and right quadrants. The whole process checks all possible radii for the iris and selects the one that exhibits the most abrupt change in average brightness, indicating the boundary where the darker iris meets the bright sclera. First, the average brightness for a specific circular contour C is calculated using the formula:

$$L(r) = \frac{1}{2\pi r} \oint_C I(x, y) ds,$$

where r is the selected radius and $I(x, y)$ is the pixel's brightness. Then, the radial derivative of this average brightness is approximated using the central difference method. The optimal iris radius is the one that maximizes this rate of change:

$$r_{best} = \arg\max_r \left(\frac{\partial L(r)}{\partial r} \right) \approx \arg\max_r (L(r + \Delta r) - L(r - \Delta r)),$$

where Δr is the finite difference step size. In this implementation, Δr is set to 3 pixels to suppress local image noise and prevent false detections. The effects of this procedure are shown in Figure 5.

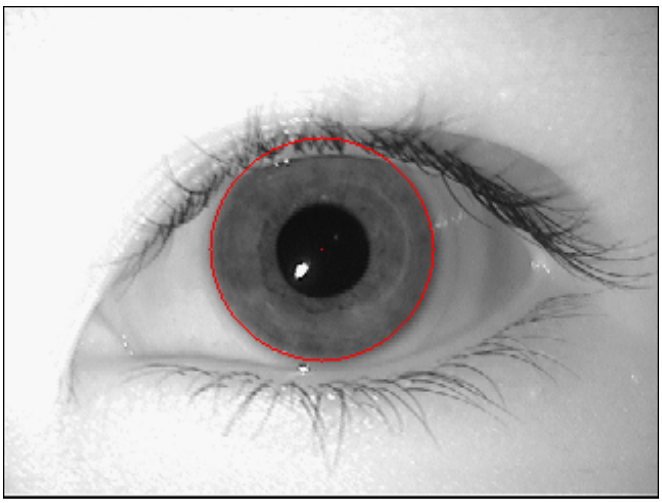

Figure 5: Calculated iris boundaries and the eye center.

2.7 Visualize both boundaries

This step visualizes both the pupil and iris boundaries, along with their shared center, overlaid on the original image. An example is shown in Figure 6.

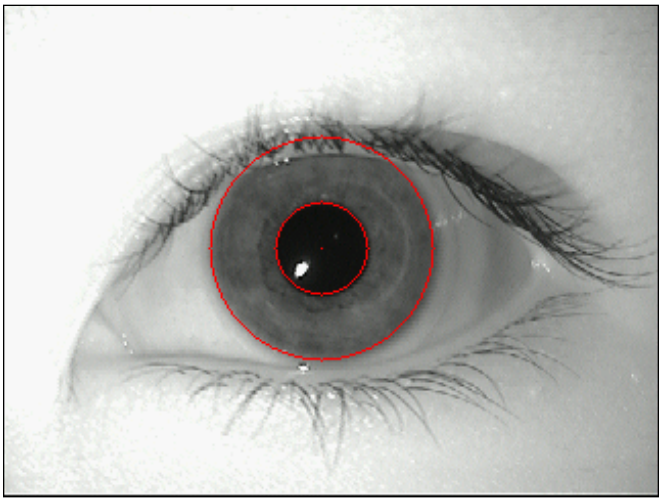

Figure 6: The eye center, pupil and iris boundaries on the original image.

2.8 Unwrap the iris to a rectangular block

The final preparation step involves transforming the annular iris region into a normalized rectangular block of fixed dimensions (64x512 pixels). This process, based on Daugman's Rubber Sheet Model, converts the iris texture from polar to Cartesian coordinates. The horizontal axis of the new image represents the angle θ (from 0 to 2π), while the vertical axis represents the radial distance from the pupil boundary to the iris boundary. For each pixel in the 64x512 rectangle, the corresponding radius and angle are calculated and mapped back to the original image's coordinates using trigonometric functions. This geometric transformation normalizes irises of varying sizes into a consistent format, as shown in Figure 7.

Figure 7: The rectangular iris block generated from the example image.

2.9 Generate iris code

The most important phase of the iris recognition process is extracting the unique biometric features from the unwrapped iris rectangle. This is achieved using Daugman Algorithm, which relies on first compressing the 2D iris rectangle into 1D strips, and then using Gabor wavelet transformations to evaluate the structural texture of the eye. The goal of this process is to translate the visual characteristics of the iris into binary code, which can later be compared against other such codes; this allows for generalising iris features between various images of the same iris, while at the same time differentiating between different irises. To accomplish this, the unwrapped iris image is horizontally divided into 8 bands, each of them representing a concentric band around the pupil of the eye. Each of these bands is then processed using the following steps.

2.9.1 Noise Mask Generation

The first step of the algorithm is cutting out the parts of the unwrapped rectangle which are likely to include elements other than the iris, such as the eyelids or eyelashes; this step makes the final code less dependent on the image acquisition technique, ie. on the particular circumstances of taking a particular iris picture. For this, the Daugman Algorithm suggests a simple solution: only some parts of each band are taken into account when generating and comparing codes. From the inner four bands, we exclude a 30° angle at the bottom of the iris; the next two bands include two symmetrically-placed sectors on the left and right side of the iris, each of them 113° wide; the last two bands only include such sectors with a width of 180° each.

In order to avoid distorting the shape of the rectangle or having to use some kind interpolation in order to approximate the ignored parts of the unwrapped rectangle, our implementation uses a static boolean mask, which is generated alongside the code and which contains information on the iris areas to be ignored when comparing the final codes.

Any calculations performed inside these masked areas are mathematically flattened and later ignored during the comparison phase.

2.9.2 Radial Averaging

In order to optimize the computation process, each 8-pixel-high band is compressed into a 1D signal using radial averaging. This approach allows us to not lose much information about the iris, as iris features tend to only differ by small amounts along radial lines. In order to accomplish such a flattening, instead of taking a simple average of the 8 pixels in a column, the algorithm applies a vertical Gaussian window. This means pixels closer to the center of the band are assigned higher weights (e.g. 0.30), while the pixels near the edges are given much lower weights (e.g. 0.03).

Once this 1D signal is generated, the algorithm calculates its average brightness. This is important, because the global brightness of an eye can vary massively depending on lighting conditions. A large average brightness score could in turn completely drown out the tiny local fluctuations representing the actual iris texture, which means it is also an important factor to control for. To solve this, the mean intensity of all valid pixels within the band is subtracted from the signal, which forces the signal to fluctuate tightly around zero, allowing the subsequent mathematical filters to strictly process the high-frequency texture shifts (the actual pattern of the iris) rather than the overall lighting of the room.

2.9.3 Gabor Wavelet Transform

Each of the 1D bands is later decomposed using Gabor wavelets, which are defined using the formula

$$G(x, x_0, f, \sigma) = \exp\left(-\frac{(x - x_0)^2}{\sigma^2}\right) \cdot \exp(-i2\pi f x),$$

where x_0 determines the wavelet location, σ controls the width of the Gaussian window, and f is the frequency of the sinusoid. In this case, we also assume the relationship between the wavelet's frequency and the width of the Gaussian window, which takes the form of

$$\sigma = \frac{1}{2}\pi f.$$

In the end, we use 128 of such Gabor wavelets evenly distributed along the width of the considered iris band; each of those has a frequency of $\frac{1}{16}$. Through that process, we end up with 128 complex coefficients per band, each of them representing the amplitude and the spatial phase shift of the local iris texture compared to the base function.

2.9.4 Phase Quantization

The final part of the code generation step is converting the response of the Gabor filter into binary code, using is a process known as phase quantization. The coefficients determined in the previous part of the algorithm determine the similarity between some given part of the image and one of the Gabor wavelet functions; in this next step, we only take into account their phase shift, since that's the part that actually describes the structure of the iris.

Importantly, instead of recording the exact numerical values produced by the real and imaginary calculations, the algorithm only records their mathematical sign. If the real part evaluates to a value greater than or equal to zero, it is recorded as a '1' (true); if it is negative, it is recorded as a '0' (false), and the exact same check is also applied to the imaginary part. Since the algorithm generates two bits (real and imaginary) for each of the 128 intervals across 8 bands, the output consists of a total of $2 \times 128 \times 8 = 2048$ bits.

2.9.5 Visualization of the Final Code

This final code is also represented visually on the screen, below the actual iris picture based on which the code has been generated, as shown in Figure 8. The boolean values are represented by black-and-white pixels; the masked parts of the unwrapped rectangle are signified by grey bits.

Figure 8: The unwrapped iris and its iris code.

2.10 Save iris code

After finishing the iris code generation, the user can export the generated code to be saved locally on their device; this allows for future comparisons of different iris codes. To streamline the process, the application automatically suggests a filename based on the originally loaded image (e.g., loading `aeval1.bmp` will prompt the user to save `aeval1_code.png`)

3 Iris Comparison

The second section of our application allows for comparing two iris pictures using their iris codes. Initially, the user can choose to work with a set of two default testing images or import chosen photos. Once the images are loaded, the process can be executed sequentially.

3.1 Calculate iris codes

First, the user is able to calculate the iris codes of both the irises in the pictures; as opposed to the Iris Recognition part of the app, here the entire process is condensed into a single button, allowing the user to quickly generate codes. The outcome of this part on an example set of images can be seen on Figure 9.

Figure 9: Two iris images displayed side by side, along with their iris codes.

3.2 Compare iris codes using Hamming Distance

After generating the codes, the user can then directly compare them in order to determine whether the chosen pictures contain the same eye. There are two algorithms available in this part.

1. **Calculate Hamming Distance (regular)** calculates the fractional Hamming Distance between the two iris codes, ie. the Hamming Distance between the unmasked parts of both the iris codes. If the Hamming Distance is lower than the experimentally determined limit value of 0.32, the images are classified as representations of the same eye; in the opposite case, the program determines that the displayed eyes are most likely different.
2. **Calculate Hamming Distance (with iris rotation)** is a modification of the above algorithm. In this case, we consider 7 different rotations of one of the irises, and compare the code of each such rotation to the code of the other iris. The result of this process is the minimum Hamming Distance obtained between one of the rotated versions of the first iris and the un-rotated version of the second one; this leads to a result that is less dependent on the specifics of the image acquisition process, which might normally result in slight eye rotations which could interfere with the comparison results (thus artificially inflating the Hamming Distance).

After the user chooses one of these variants, a pop-up appears, which states the calculated Hamming Distance and the conclusion regarding the iris comparison.

4 Conclusion

In conclusion, the iris recognition application implements a complete pipeline for biometric identification based on standard eye photographs. It also breaks down the complex recognition process into individual, visualizable steps, which allows the user to understand each step of the process and potentially adjust its parameters (if such a need arises).

The Iris Comparison part allows to test this framework empirically on the MMU Iris Dataset, further demonstrating its reliability.